Diffie-Hellman Key Exchange

AIM: *To demonstrate how two parties can securely exchange secret keys over an insecure communication channel using the Diffie-Hellman key exchange algorithm*

DESCRIPTION:

The Diffie-Hellman (DH) key exchange algorithm allows two parties, commonly called Alice and Bob, to establish a common secret key over an insecure communication channel.

The important point is that the secret/private values are never transmitted. Only the public parameters and public keys are exchanged.

The shared secret can then be used as a key for symmetric encryption algorithms such as AES.

IMPLEMENTATION:

```
p = 23
g = 5
a = 6
b = 15
A = pow(g, a, p)
B = pow(g, b, p)
print("Public Parameters:")
print("Prime (p):", p)
print("Generator (g):", g)
print("\nAlice:")
print("Private Key:", a)
print("Public Key:", A)
print("\nBob:")
print("Private Key:", b)
print("Public Key:", B)
alice_secret = pow(B, a, p)
bob_secret = pow(A, b, p)
print("\nShared Secret:")
print("Alice's Shared Secret:", alice_secret)
print("Bob's Shared Secret:", bob_secret)
if alice_secret == bob_secret:
    print("\nKey Exchange Successful!")
    print("Both parties have the same secret key.")
else: print("\nKey Exchange Failed!")
```

RESULT:

```
Public Parameters:
Prime (p): 23
Generator (g): 5

Alice:
Private Key: 6
Public Key: 8

Bob:
Private Key: 15
Public Key: 19

Shared Secret:
Alice's Shared Secret: 2
Bob's Shared Secret: 2

Key Exchange Successful!
Both parties have the same secret key.
```

```
Public Parameters:
Prime (p): 29
Generator (g): 2

Alice:
Private Key: 7
Public Key: 12

Bob:
Private Key: 11
Public Key: 18

Shared Secret:
Alice's Shared Secret: 17
Bob's Shared Secret: 17

Key Exchange Successful!
Both parties have the same secret key.
```

INSTITUTE OF TECHNOLOGY
స్వయం తేజస్విన్ భవ
1979

CONCLUSION:

Diffie-Hellman demonstrates how two parties can establish a common secret key over an insecure communication channel. The security of the scheme is based on the computational difficulty of the discrete logarithm problem. The resulting shared secret can subsequently be used with a symmetric encryption algorithm for secure communication.

Secure Hash Algorithm 256

AIM: *To implementation of SHA-256 cryptographic hash function.*

DESCRIPTION:

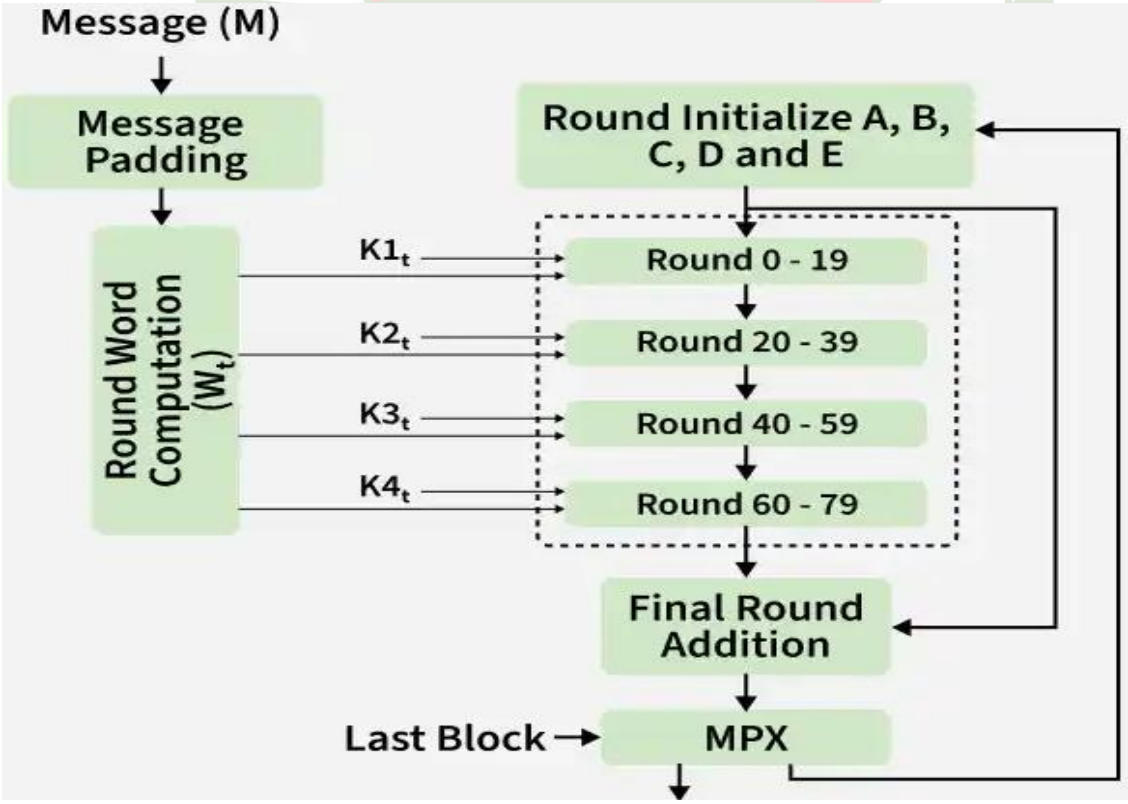
SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function belonging to the SHA-2 family. It converts an input message of any length into a fixed-length 256-bit (32-byte) hash value, usually represented as a 64-character hexadecimal string.

SHA-256 is a one-way function, meaning that it is computationally infeasible to recover the original message from its hash. Even a small change in the input produces a significantly different hash value.

Properties

- ***Input:*** *Data of any length*
- ***Output:*** *256-bit hash*
- ***Hexadecimal representation:*** *64 characters*
- ***Deterministic:*** *Same input always produces the same hash*
- ***Avalanche effect:*** *Small input changes produce a very different hash*
- ***One-way:*** *Original data cannot practically be derived from the hash*

FLOWCHART:



IMPLEMENTATION:

```
import hashlib
message = input("Enter the message: ")
hash_value = hashlib.sha256(message.encode()).hexdigest()
print("Original Message:", message)
print("SHA-256 Hash:", hash_value)
```

RESULT:

```
Enter the message: Hello World
Original Message: Hello World
SHA-256 Hash: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b...
```

```
Enter the message: Hello World
Original Message: Hello World
SHA-256 Hash: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b...
```

CONCLUSION:

Thus, the SHA-256 cryptographic hash function was successfully implemented and tested. The experiment demonstrated how an input message can be converted into a fixed-size 256-bit hash value. SHA-256 provides strong data integrity and is widely used in security applications such as digital signatures, password protection, blockchain technology, and file verification..

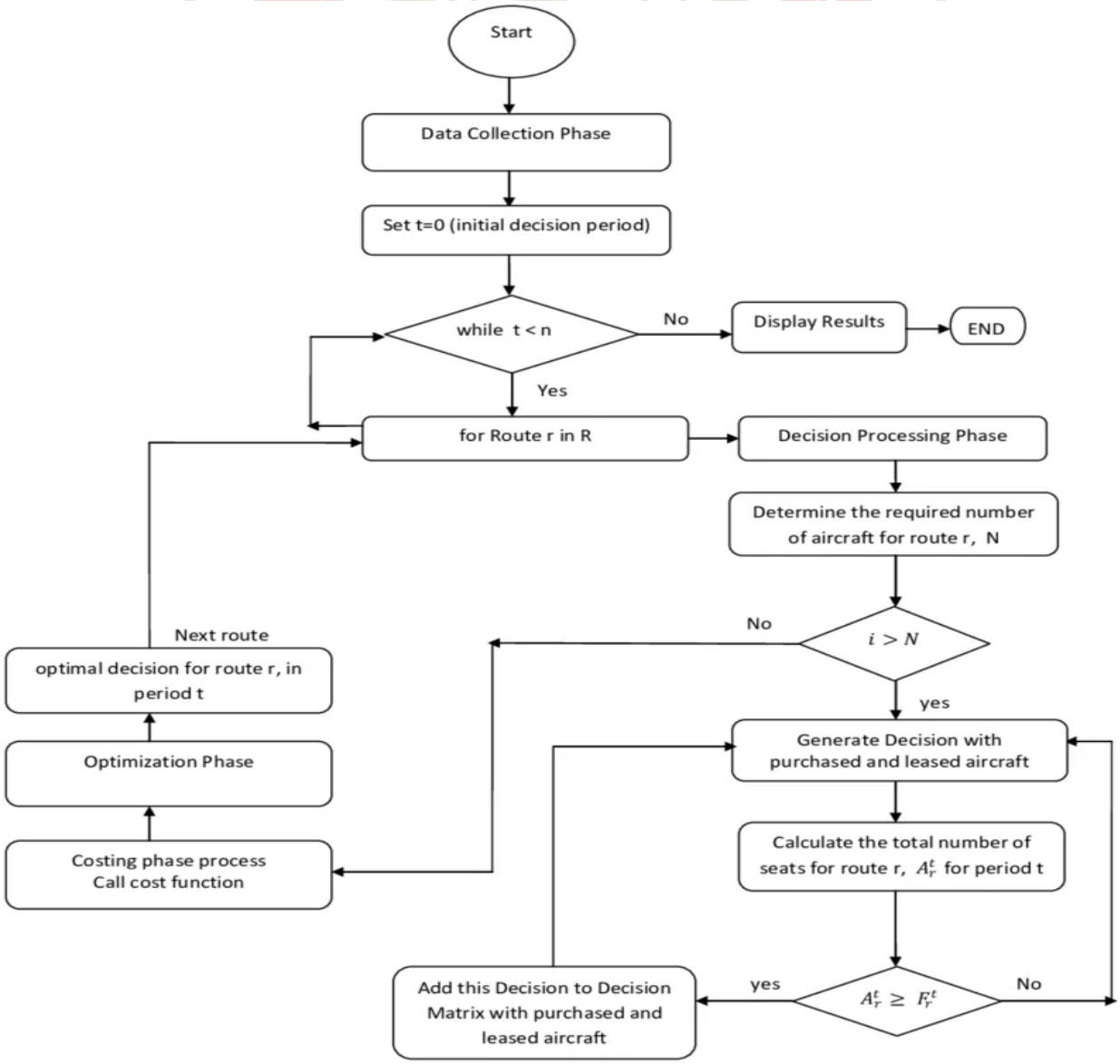
Digital Signature Standard

AIM: *To implementation, generating and verifying of Digital Signature Standard (DSS).*

DESCRIPTION:

The Digital Signature Standard (DSS) is a standard developed by the National Institute of Standards and Technology (NIST) for generating and verifying digital signatures. It is used to provide authentication, data integrity, and non-repudiation in digital communication. DSS uses cryptographic algorithms, such as the Digital Signature Algorithm (DSA), to create a unique digital signature for a message. The sender generates the signature using a private key, while the receiver verifies it using the corresponding public key. If the message is modified after signing, the verification process fails, indicating that the message is not authentic or has been altered.

FLOWCHART:



IMPLEMENTATION:

```
from cryptography.hazmat.primitives.asymmetric import dsa
from cryptography.hazmat.primitives import hashes
from cryptography.exceptions import InvalidSignature
private_key = dsa.generate_private_key(key_size=2048)
public_key = private_key.public_key()
print("DSA key pair generated successfully.")
message = b"Digital Signature Standard"
signature = private_key.sign(message,hashes.SHA256())
print("\nOriginal Message:")
print(message.decode())
print("\nDigital Signature:")
print(signature.hex())
try:
    public_key.verify(signature,message,hashes.SHA256())
    print("\nSignature Verification:")
    print("Signature is VALID.")
except InvalidSignature:
    print("\nSignature Verification:")
    print("Signature is INVALID.")
modified_message = b"Digital Signature Standard Modified"
try:
    public_key.verify(signature,modified_message,hashes.SHA256())
    print("\nModified Message Verification:")
    print("Signature is VALID.")
except InvalidSignature:
    print("\nModified Message Verification:")
    print("Signature is INVALID.")
```

RESULT:

INSTITUTE OF TECHNOLOGY

Original Message: Secure Digital Communication
Digital Signature: 3046022100...
Signature Verification: Signature is VALID.
Modified Message Verification: Signature is INVALID.

Original Message: Data Integrity and Authentication
Digital Signature: 3045022100...
Signature Verification: Signature is VALID.
Modified Message Verification: Signature is INVALID.

CONCLUSION:

The experiment demonstrates that digital signatures can be used to verify the authenticity and integrity of digital messages. The private key is used to create the signature, while the corresponding public key is used for verification. Any modification to the signed message results in verification failure, making digital signatures an important component of secure digital communication.

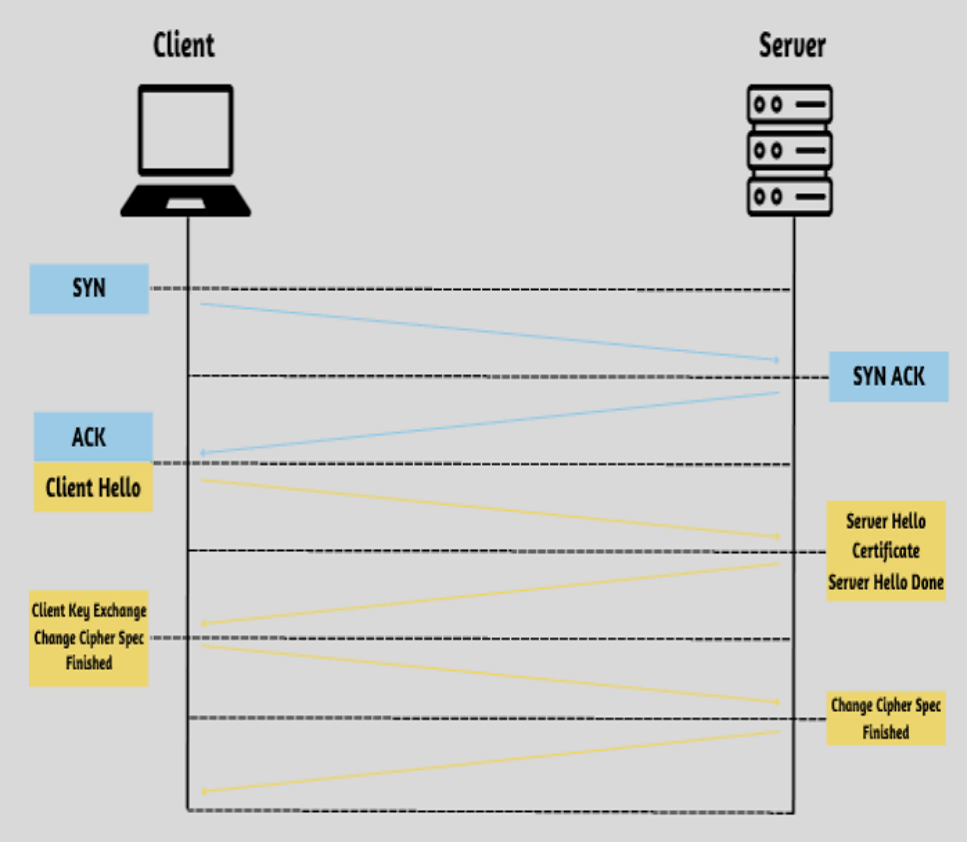
Secure Socket Layer

AIM: *To implement Secure Socket Layer (SSL) for secure communication over a network and establish an encrypted client-server connection using SSL/TLS certificates.*

DESCRIPTION:

Secure Socket Layer (SSL) is a protocol designed to provide secure communication between a client and a server over an insecure network. SSL uses encryption, authentication, and integrity mechanisms to protect data transmitted between communicating parties. During the SSL/TLS handshake, the client and server negotiate security parameters and establish a secure session. A server certificate is used to authenticate the server and establish trust through a publickey infrastructure. After the handshake, symmetric encryption is generally used for efficient protection of application data. SSL has been superseded by Transport Layer Security (TLS), although the term SSL is still commonly used when referring to secure connections such as HTTPS.

FLOWCHART:



IMPLEMENTATION:

Server.py

```
import socket
import ssl
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain("server.crt", "server.key")
server = socket.socket()
server.bind(("localhost", 12345))
server.listen(1)
with context.wrap_socket(server, server_side=True) as ssock:
 conn, addr = ssock.accept()
 print("Secure connection established")
 data = conn.recv(1024).decode()
 print("Client:", data)
 conn.send(b"Hello from secure server")
 conn.close()
```

Client.py

```
import socket
import ssl
context = ssl.create_default_context()
context.check_hostname = False
context.verify_mode = ssl.CERT_NONE
with socket.create_connection(("localhost", 12345)) as sock:
    with context.wrap_socket(sock, server_hostname="localhost") as ssock:
        ssock.send(b"Hello from secure client")
        print("Server:", ssock.recv(1024).decode())
```

RESULT:

Server

```
Secure connection established
Client: Hello from secure client
```

Client

```
Server: Hello from secure server
```

CONCLUSION:

SSL/TLS-based secure socket communication was successfully implemented between a client and server. The experiment demonstrated how encryption, authentication, and integrity can be provided for network communication using secure sockets

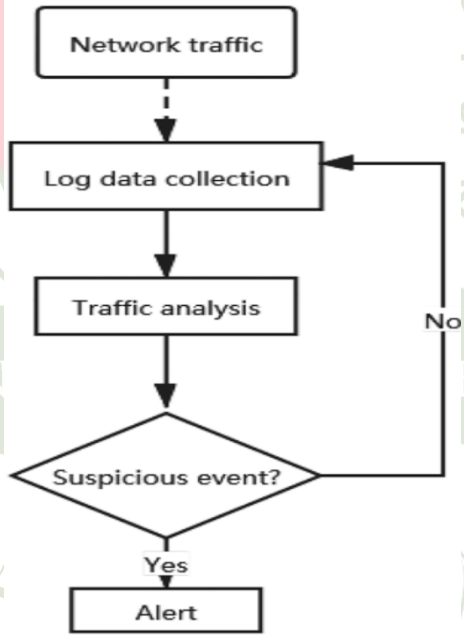
Intrusion Detection System Using Snort

AIM: *To demonstrate the working of an Intrusion Detection System (IDS) using Snort and to detect and monitor suspicious network traffic using predefined security rules.*

DESCRIPTION:

An Intrusion Detection System (IDS) is a security mechanism that monitors network traffic or system activities to identify suspicious behaviour and potential security violations. Snort is an open-source network intrusion detection and prevention tool that analyses packets using predefined rules. It supports signature-based detection, where network traffic is compared against known patterns of attacks or suspicious activities. Snort can generate alerts when traffic matches a configured rule, helping administrators investigate potential threats. An IDS primarily detects and reports suspicious activity, whereas an Intrusion Prevention System (IPS) can also block traffic.

FLOWCHART:



IMPLEMENTATION:

Verify snort installation:

```
  ,,_     -*> Snort! <*-
 o"  )~   Version 2.9.20 GRE (Build 82) x86_64
  ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
          Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
          Copyright (C) 1998-2013 Sourcefire, Inc., et al.
          Using libpcap version 1.10.4 (with TPACKET_V3)
          Using PCRE version: 8.39 2016-06-14
          Using ZLIB version: 1.3
```

Create the Detection Rule:

```
  GNU nano 7.2                                  /etc/snort/rules/local.rules *
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# ----------------
# LOCAL RULES
# ----------------
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert icmp any any -> any any (msg:"ICMP Ping Detected"; itype:8; sid:1000001; rev:1;)
```

Generate Test Traffic (while Snort is running):

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.051 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.064 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.087 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.069 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2999ms
rtt min/avg/max/mdev = 0.051/0.067/0.087/0.012 ms
```

Start Snort:

```
10/14-14:06:04.407814  [**] [1:1000001:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 127.0.0.1 -> 127.0.0.1
10/14-14:06:04.407814  [**] [1:528:5] BAD-TRAFFIC loopback traffic [**] [Classification: Potentially Bad Traffic] [Prior
ity: 2] {ICMP} 127.0.0.1 -> 127.0.0.1
10/14-14:06:04.407814  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority:
 2] {ICMP} 127.0.0.1 -> 127.0.0.1
10/14-14:06:04.407827  [**] [1:528:5] BAD-TRAFFIC loopback traffic [**] [Classification: Potentially Bad Traffic] [Prior
ity: 2] {ICMP} 127.0.0.1 -> 127.0.0.1
10/14-14:06:04.407827  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority:
 2] {ICMP} 127.0.0.1 -> 127.0.0.1
```

CONCLUSION:

The Snort Intrusion Detection System was configured to detect a specific network traffic pattern and generate alerts. The experiment demonstrated the role of IDS rules in monitoring network traffic and identifying potentially suspicious activity.